KOI Content Pack

Marketplace edition, version 1.2.3

Customer Install & Usage Guide

Document	KOI Content Pack — Customer Install & Usage Guide
Pack	KOI (the pack published in the Cortex Marketplace)
Pack version	1.2.3
Source of record	demisto/content / Packs/Koi/Integrations/Koi/Koi.yml
Source md5	5497cdddedeb0c0d7d0b371aa075a64c
Integration id / display	KOI / KOI
Category	Endpoint
Support / author	xsoar / Cortex XSOAR
Ships	Integration only — 13 commands, 67 arguments, 131 output declarations
Applies to	Cortex XSIAM (event collection + commands) and Cortex XSOAR (commands only)
Optional companion pack	KOI Content Extension (KoiContentExtension, v1.0.0) — parsing rules, modeling rules, 10 playbooks and a dashboard. SEPARATE, additional content; NOT part of this Marketplace pack (see 1.3).
Document date	20 July 2026

This is the MARKETPLACE KOI pack, not a custom KOI pack

There is more than one pack called `KOI`. They share the pack name, the integration id `KOI`, the category `Endpoint`, the author "Cortex XSOAR" and the `koi-*` command namespace, so they are indistinguishable at a glance.

This guide describes ONLY the pack identified in the table above: demisto/content / Packs/Koi/Integrations/Koi/Koi.yml, version 1.2.3, 13 commands, integration only. Verify the version in the Marketplace before applying anything in this document.

Table of Contents

1. About This Pack

1.1 What the pack is

The KOI pack connects Cortex to the KOI API. It contains a single content item: the KOI integration. The integration does two things — it collects KOI events into Cortex XSIAM, and it exposes 13 commands for querying KOI inventory and for reading and changing KOI governance objects (policies, allowlist, blocklist).

Property	Value
Pack name	KOI
Pack version	1.2.3
Pack created	2026-03-26T00:00:00Z
Support level	xsoar
Author	Cortex XSOAR
Pack categories	Endpoint
Marketplaces	xsoar, marketplacev2, platform
Integration id	KOI
Integration display name	KOI
Integration category	Endpoint
Minimum Cortex version (fromversion)	6.10.0
Docker image	demisto/fastapi:0.125.0.10158186
Event collector (isfetchevents)	true, but overridden to false for xsoar — event collection is XSIAM / platform only
Commands	13
Command arguments	67
Declared outputs	131 declarations across 68 distinct context paths

1.2 Collision warning: this pack cannot coexist with a custom KOI pack

One tenant, one KOI pack

A separate, privately built KOI pack exists (a different implementation, with a larger command set). It uses the **same pack name (KOI)**, the **same integration id (KOI)**, the **same category (Endpoint)** and the same `koi-*` command names.

Because Cortex keys content on those identifiers, the two packs cannot be installed side by side. Installing one over the other overwrites it: the integration definition, its commands and its instance configuration are replaced.

Practical consequences: any playbook, automation or query built against the other pack's commands or context paths breaks silently at the point where a command it calls no longer exists. Decide which pack a tenant runs, and record that decision. If you are unsure which pack a tenant currently has, compare the command list in section 6 of this guide against the commands the installed integration actually offers.

1.3 What ships in the pack — and what does not

This pack is an integration and nothing else. There are no playbooks, no automations, no parsing rules, no modeling rules, no dashboards, no layouts and no incident/alert types.

Content type	In this pack	What that means for you
Integration	Yes (1)	The KOI integration: 13 commands on every platform, plus an event collector that is enabled on Cortex XSIAM only (section 2.1).
Playbooks	No	No triage, investigation or response automation ships with the pack. Any automation is yours to build.
Automations / scripts	No	No helper scripts are installed.
Parsing rules	No	Collected events are stored exactly as the collector sends them. No field promotion, no reshaping.
Modeling rules	No	Events are NOT normalised to XDM. No Cortex Data Model field is populated by this pack.
Dashboards / widgets	No	No KOI dashboard is installed. Build your own on the raw dataset (section 8).
Layouts, incident/alert types	No	KOI events are not turned into typed alerts by this pack.

Events are not normalised

Because the pack ships no parsing rule and no modeling rule, KOI events land raw. Verified on the tenant: no XDM field is populated at all — every query, correlation rule, dashboard widget and report you build must target the raw KOI field names shown in section 8. Content that assumes XDM (for example, generic XDM-based correlation rules) will not match KOI data.

Sources: that the pack ships no parsing rule and no modeling rule is from inspecting the pack's own contents; that no XDM field is populated is from VERIFIED_FACTS.md §3.2 (live, 20 July 2026).

An optional companion pack adds the missing content — as SEPARATE content

Everything this pack lacks — parsing rules, modeling rules, playbooks and a dashboard — is available in a separate, optional companion pack, **KOI Content Extension** (pack folder `KoiContentExtension`, currentVersion 1.0.0, community support). It ships **no integration and no commands of its own**: it normalises and models the `koi_koi_raw` dataset this integration produces, and adds 10 investigation and response playbooks built strictly against the 13 commands documented here, plus one alerts dashboard.

It is additional content and is NOT part of the Marketplace KOI pack described in this guide. It declares the KOI pack as a mandatory dependency, so this Marketplace pack must be installed first. Install the companion pack only if you want the normalisation and playbooks; nothing in this guide requires it.

Source: the companion pack's own pack_metadata.json and its contents under Packs/KoiContentExtension/. This is outside the VERIFIED_FACTS evidence base for the Marketplace pack.

2. Requirements

2.1 Platform

Requirement	Value
Minimum Cortex version	`6.10.0 (the integration's fromversion)`
Docker image	`demisto/fastapi:0.125.0.10158186`
Event collection	`Cortex XSIAM / platform only. The YAML sets isfetchevents = true and then overrides it with isfetchevents:xsoar = false; every Collect-section parameter is additionally hidden on xsoar. On an XSOAR tenant the pack is a command integration and nothing else.`
Marketplaces the pack targets	`xsoar, marketplacev2, platform — note that xsoar is still listed even though the collector is disabled there`

On XSOAR this pack collects no events at all

The pack advertises an event collector, but it is switched off for the xsoar marketplace at the source level: `isfetchevents: true` is immediately followed by `isfetchevents:xsoar: false`, and the Collect parameters carry `hidden: [xsoar]`.

Everything in section 8 of this guide — the dataset, the schemas, the XQL — applies to Cortex XSIAM only. On XSOAR you get the commands and nothing else.

Source: the pinned pack source (lines 990–991) and VERIFIED_FACTS.md §2.

2.2 KOI API access

- **An API key for your KOI tenant.** The pack's own parameter help for `api_key` reads: "The API key for authenticating with the KOI API. See the help section for instructions on creating an API key."
- Authentication is a bearer token: the integration sends `Authorization: Bearer <api_key>`.

- API base: `https://api.prod.koi.security` with the path prefix `/api/external/v2`. The default value of the Server URL parameter is `https://api.prod.koi.security/`.

Source: VERIFIED_FACTS.md §4.1 (endpoint map read from the integration's Koi.py) and marketplace-pack.json (parameter defaults).

2.3 Network path

Whatever executes the integration must reach the KOI API over outbound HTTPS (TCP 443) to api.prod.koi.security. That executor is either the Cortex tenant itself or a Cortex engine.

> **If you run through an engine, test from the engine**
>
> On the reference deployment both KOI instances run through a Cortex engine rather than direct tenant egress. When an engine is in the path, a connectivity test performed from the tenant proves nothing: the traffic does not take that route. Test reachability from the engine host.
>
> *Source: VERIFIED_FACTS.md §2 (live).*

3. Installing the Pack

The pack is published in the Cortex Marketplace and is installed from there. No SDK, no zip and no manual item import is required.

1. **Confirm no conflicting KOI pack is present.** See section 1.2. If a different KOI pack (same name, same integration id) is installed, installing this one replaces it.
2. **Open the Marketplace** in your Cortex tenant and search for `KOI`.
3. **Check the version.** This guide describes version 1.2.3. If the Marketplace offers a newer version, re-check the command list and parameters against the release notes before relying on this document.
4. **Install the pack,** then confirm that the `KOI` integration appears in the integrations list and that nothing else did — no playbooks, no rules, no dashboards (section 1.3).

Navigation labels ↑ are Cortex UI wording and are not part of the pack sources; they were not re-verified for this document. Everything about the pack itself (name, version, contents) is from the pinned source.

4. Configuring an Integration Instance

The integration declares 9 configuration parameters. The table below is generated from the pack source, so it lists every parameter the integration actually has — including the ones that appear only on Cortex XSIAM.

4.1 All configuration parameters

Parameter name	Display name	Required	Default	Purpose
`url`	Server URL	Yes	https:// api.prod.koi.security/	The KOI API server URL.

Parameter name	Display name	Required	Default	Purpose
`api_key`	API Key	Yes	—	The API key for authenticating with the KOI API. See the help section for instructions on creating an API key.
`insecure`	Trust any certificate (not secure)	No	—	(no help text in the pack)
`proxy`	Use system proxy settings	No	—	(no help text in the pack)
`isFetchEvents`	Fetch events	No	—	(no help text in the pack)
`event_types_to_fetch`	Fetch event types	Yes	Alerts,Audit	Select which event types to fetch from KOI.
`audit_types_filter`	Audit log type filter	No	—	Filter audit logs by type(s). If not specified, all audit log types will be fetched.
`max_fetch`	Maximum number of events per fetch	No	5000	Maximum number of events to fetch per type per fetch cycle (default: 5000).
`eventFetchInterval`	Events Fetch Interval	No	1	(no help text in the pack)

4.2 How the parameters are grouped and typed

Parameter	Section	Type	Advanced	Hidden on
`url`	Connect	Short text (0)	No	—
`api_key`	Connect	Encrypted (credential) (14)	No	—
`insecure`	Connect	Boolean (8)	Yes	—
`proxy`	Connect	Boolean (8)	Yes	—
`isFetchEvents`	Collect	Boolean (8)	No	xsoar
`event_types_to_fetch`	Collect	Multi select (16)	No	xsoar
`audit_types_filter`	Collect	Multi select (16)	No	xsoar
`max_fetch`	Collect	Short text (0)	No	xsoar
`eventFetchInterval`	Collect	Events fetch interval (19)	Yes	xsoar

Parameters hidden on the xsoar marketplace are the event-collection parameters: they are shown on Cortex XSIAM only. Together with the isfetchevents:xsoar override (section 2.1), this means an XSOAR instance has no collector to configure.

4.3 A known-good configuration

The configuration below was in use on both instances of the reference tenant on 20 July 2026, and is a reasonable starting point:

Parameter	Value in use
url (Server URL)	`https://api.prod.koi.security/`
isFetchEvents (Fetch events)	`true`
event_types_to_fetch	`Alerts, Audit`
audit_types_filter	`empty (no filter — all audit types)`
max_fetch	`5000`
eventFetchInterval	`1 (minute)`
insecure / proxy	`false / false`

Source: VERIFIED_FACTS.md §2 (live).

> **Two instances, one dataset**
>
> If you configure more than one instance with event collection enabled, all of them write into the same koi_koi_raw dataset, and the pack ships no field identifying which instance produced a row. On the reference tenant, two instances (KOI_PAET and KOI_PLTS) fetch Alerts and Audit into that one dataset. If you need to tell the sources apart, plan for it before you turn on the second collector.
>
> *Source: VERIFIED_FACTS.md §2 (live).*

5. Verifying the Instance

1. **Run the instance Test.** This exercises the Server URL and the API key.
2. **Run a read-only command from the CLI / War Room,** for example:

```
!koi-policy-list limit=5
!koi-inventory-list limit=5
```

3. **If event collection is enabled,** wait one fetch interval and query the dataset (section 8).

What could not be verified for this document

The API behind 8 of the 13 commands — not all of them — was exercised one layer down, by direct request to the KOI API endpoints listed in section 6, using the same bearer authentication and the same API keys the instances use. These are the commands whose endpoints that covers:

`koi-policy-list, koi-allowlist-get, koi-blocklist-get, koi-inventory-list, koi-inventory-item-get, koi-inventory-item-endpoints-list, koi-inventory-search, koi-get-events`

The remaining 5 were deliberately NOT run, because every one of them changes state in your KOI tenant:

`koi-policy-status-update, koi-allowlist-items-add, koi-allowlist-items-remove, koi-blocklist-items-add, koi-blocklist-items-remove`

Nothing in this guide describes the observed behaviour of those five. What is said about them comes from the pack's own files alone — the YAML for what they declare, Koi.py for the endpoints they would call.

No command was executed through the Cortex tenant used for verification, because that tenant rejects the API paths the tooling needs (investigation search returns a redirect, incident creation returns an empty body, and an API-key user's playground is created malformed so every command run there panics). No War Room output and no context mapping was ever observed.

Established by the API-level testing: those endpoints exist, the parameters are accepted or rejected as documented, and the response shapes are as recorded. NOT established: the Cortex-side context mapping and human-readable output of any command — those come from the pack's own files (the YAML for what is declared, Koi.py for what the code does) and are presented as such throughout this guide.

Source: VERIFIED_FACTS.md §1.1 and §6.

6. Command Reference

Every table in this section is generated from the pinned pack source (Packs/Koi/Integrations/Koi/Koi.yml, md5 5497cdddedeb0c0d7d0b371aa075a64c). Nothing here is transcribed by hand. The pack declares 13 commands, 67 arguments and 131 outputs.

Commands that are NOT in this pack

The other KOI pack has a larger command set. These names do not exist here and will fail with "unknown command":

`koi-devices-list, koi-device-inventory-get, koi-koidex-risk-report, koi-koidex-search, koi-remediations-list, koi-approval-requests-list, koi-findings-list, koi-users-list, koi-groups-list, koi-runtime-policies-list, koi-runtime-policy-get, koi-fetch-context-get, koi-fetch-context-set`

There is likewise no Koi.Device.* context prefix in this pack (section 7).

6.1 Command index

Command	Changes state (and is it flagged?)	Args	Outputs	Context prefix
`koi-get-events`	No — but not safe to repeat; can write to koi_koi_raw (9.5)	5	4	KOI.Event
`koi-policy-list`	No — read-only, repeatable	3	9	Koi.Policy
`koi-allowlist-get`	No — read-only, repeatable	0	9	Koi.Allowlist
`koi-allowlist-items-remove`	Yes — flagged execution: true	5	0	none
`koi-allowlist-items-add`	Yes — NOT flagged	5	0	none
`koi-blocklist-get`	No — read-only, repeatable	0	9	Koi.Blocklist
`koi-blocklist-items-remove`	Yes — flagged execution: true	5	0	none
`koi-blocklist-items-add`	Yes — NOT flagged	5	0	none
`koi-policy-status-update`	Yes — NOT flagged	2	9	Koi.Policy
`koi-inventory-list`	No — read-only, repeatable	21	27	Koi.Inventory
`koi-inventory-item-get`	No — read-only, repeatable	3	27	Koi.Inventory
`koi-inventory-search`	No — read-only, repeatable	7	27	Koi.Inventory
`koi-inventory-item-endpoints-list`	No — read-only, repeatable	6	10	Koi.Inventory.Endpoint

5 of the 13 commands change state in your KOI tenant, but only 2 are flagged execution: true in the pack source — koi-allowlist-items-remove and koi-blocklist-items-remove. Cortex treats those two as potentially harmful and will require confirmation or explicit permission to run them from a playbook. The other 3 mutate governance state with no such flag and run without that guard: `koi-policy-status-update, koi-allowlist-items-add, koi-blocklist-items-add`. Treat all 5 as state-changing regardless of the flag — "not flagged" means not flagged, not safe. The remaining 8 commands change nothing in your KOI tenant, but only 7 of them are freely repeatable.

`koi-get-events` is the exception, and the row above says so. **It is a GET and it changes nothing in KOI, but it is not safe to run repeatedly**: the pack's own description of it says it is for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism. Its `should_push_events` argument writes the events it retrieves into the `koi_koi_raw` dataset when set to true, alongside whatever the collector is already fetching. Leave it at its default and do not put this command in a loop, a scheduled job, or a "safe to re-run" runbook step. See 9.5.

API endpoints quoted per command below are read from the integration's own source code, Koi.py (VERIFIED_FACTS.md §4.1) — that is the pack's source code, not its YAML and not by itself a live observation. The map has 11 entries; 8 of them were additionally exercised by direct API request, and the three behind the 5 state-changing commands (PUT /policies/{id}, POST and DELETE on /policies/allowlist, POST and DELETE on /policies/blocklist) were never called. Everything else per command is from the pack source.

6.2 koi-get-events

> **Read-only against KOI, but NOT safe to repeat**
>
> This command changes nothing in your KOI tenant, so it is not one of the five state-changing commands. It is still the one non-mutating command you should not re-run casually: the pack's own description says it is for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism.
>
> `should_push_events` writes the retrieved events into the `koi_koi_raw` dataset when set to true — the same dataset the collector writes to. Leave it at its default. Use the event collector for ingestion; use this command only to look at what the API returns.
>
> *Source: the description is from the pack source; the caveat is recorded in VERIFIED_FACTS.md §1.1.*

Gets events from KOI. Use this command for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism.

API endpoint: `GET /alerts and GET /audit-logs` · **Context prefix:** `KOI.Event`

Arguments

Argument	Required	Default	Predefined values	Description
`event_type (list)`	No	Alerts,Audit	Alerts, Audit	The type of events to retrieve. If not specified, uses the value configured in the integration parameters.
`limit`	No	50	—	The maximum number of events to return per type.
`start_time`	No	—	—	Filter events created at or after this time. Supports ISO 8601 format or relative time expressions (e.g., "3 days ago", "2024-01-01T00:00:00Z").
`end_time`	No	—	—	Filter events created at or before this time. Supports ISO 8601 format or relative time expressions (e.g., "now", "2024-01-01T00:00:00Z").
`should_push_events`	No	false	true, false	The flag that indicates whether to push events to Cortex XSIAM. Pushing events is supported on Cortex XSIAM only. When set to false, or on non-XSIAM platforms, events are displayed without being pushed.

Outputs

Context path	Type	Description
KOI.Event.id	String	The unique identifier of the event.
KOI.Event.source_log_type	String	The source log type of the event (Alerts or Audit).
KOI.Event._time	Date	The timestamp of the event in ISO 8601 format.
KOI.Event.created_at	Date	The creation time of the event (audit logs).

6.3 koi-policy-list

Retrieves a list of all policies. Use 'page' and 'page_size' to fetch a specific page, or use 'limit' to auto-paginate and collect up to the specified number of policies. If 'page' is provided, 'limit' is ignored.

API endpoint: `GET /policies` • **Context prefix:** `Koi.Policy`

Arguments

Argument	Required	Default	Predefined values	Description
page	No	—	—	Page number for pagination (1-based). When provided, fetches a single page and ignores the limit argument.
page_size	No	—	—	Number of results per page (default: 50, max: 500). Used only in single-page mode together with the page argument.
limit	No	50	—	Maximum total number of policies to return (default: 50, max: 1000). When provided without page, auto-paginates to collect up to this many policies.

Outputs

Context path	Type	Description
Koi.Policy.id	Number	The unique identifier of the policy.
Koi.Policy.name	String	The name of the policy.
Koi.Policy.description	String	The description of the policy.
Koi.Policy.action	String	The action taken by the policy (e.g., block).
Koi.Policy.enabled	Boolean	Whether the policy is enabled.
Koi.Policy.group_ids	Unknown	List of group IDs associated with the policy.
Koi.Policy.creator_fullname	String	The full name of the policy creator.
Koi.Policy.created_at	Date	The creation time of the policy in ISO 8601 format.
Koi.Policy.updated_at	Date	The last update time of the policy in ISO 8601 format.

6.4 koi-allowlist-get

Retrieves all items in the allowlist.

API endpoint: `GET /policies/allowlist` · **Context prefix:** `Koi.Allowlist`

This command takes no arguments.

Outputs

Context path	Type	Description
`Koi.Allowlist.item_id`	String	The unique identifier of the allowlist item.
`Koi.Allowlist.item_name`	String	The name of the allowlist item.
`Koi.Allowlist.item_display_name`	String	The display name of the allowlist item.
`Koi.Allowlist.marketplace`	String	The marketplace of the allowlist item (e.g., vscode).
`Koi.Allowlist.publisher_name`	String	The publisher name of the allowlist item.
`Koi.Allowlist.package_name`	String	The package name of the allowlist item.
`Koi.Allowlist.notes`	String	Notes associated with the allowlist item.
`Koi.Allowlist.created_by`	String	The user who created the allowlist item.
`Koi.Allowlist.created_at`	Date	The creation time of the allowlist item in ISO 8601 format.

6.5 koi-allowlist-items-remove

> **Potentially harmful — flagged execution: true in the pack**
>
> This command changes state in your KOI tenant and is marked as potentially harmful in the pack source. Cortex gates it accordingly. Review the arguments before running it, and be aware that it declares no outputs, so nothing records what it removed.
>
> *It was not run during verification. Nothing in this guide describes its observed behaviour.*

Removes one or more items from the global allowlist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk removal from a JSON file.

API endpoint: `DELETE /policies/allowlist` · **Context prefix:** `none`

Arguments

Argument	Required	Default	Predefined values	Description
`item_id`	No	—	—	The ID of the item to remove from the allowlist. Required when not using items_list_raw_json_entry_id.

Argument	Required	Default	Predefined values	Description
`marketplace`	No	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The source marketplace of the item. Required when not using items_list_raw_json_entry_id.
`created_by`	No	—	—	Email of the user who created this entry.
`notes`	No	—	—	Additional notes about the removal.
`items_list_raw_json_entry_id`	No	—	—	War Room entry ID of a JSON file containing a list of items to remove. Each item must have "item_id" and "marketplace" fields. Optional fields: "created_by", "notes". When provided, item_id and marketplace arguments are ignored.

Outputs

> **This command declares no outputs**
>
> It writes nothing to the context — it returns a War Room message only. A playbook cannot branch on its result, and a subsequent command cannot read what it did. If you need to confirm the change, follow it with the matching -get command.

6.6 koi-allowlist-items-add

> **Changes state — but is NOT flagged execution: true**
>
> This command modifies governance state in your KOI tenant, and the pack does not mark it as potentially harmful. It therefore runs without the confirmation Cortex applies to flagged commands. Treat it with the same care as the flagged ones.
>
> *It was not run during verification. Nothing in this guide describes its observed behaviour.*

Adds one or more items to the global allowlist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk addition from a JSON file.

API endpoint: `POST /policies/allowlist` • **Context prefix:** `none`

Arguments

Argument	Required	Default	Predefined values	Description
item_id	No	—	—	The ID of the item to add to the allowlist. Required when not using items_list_raw_json_entry_id.
marketplace	No	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The source marketplace of the item. Required when not using items_list_raw_json_entry_id.
created_by	No	—	—	Email of the user who created this entry.
notes	No	—	—	Additional notes about the entry.
items_list_raw_json_entry_id	No	—	—	War Room entry ID of a JSON file containing a list of items to add. Each item must have "item_id" and "marketplace" fields. Optional fields: "created_by", "notes". When provided, item_id and marketplace arguments are ignored.

Outputs

> **This command declares no outputs**
>
> It writes nothing to the context — it returns a War Room message only. A playbook cannot branch on its result, and a subsequent command cannot read what it did. If you need to confirm the change, follow it with the matching -get command.

6.7 koi-blocklist-get

Retrieves all items in the blocklist.

API endpoint: `GET /policies/blocklist` · **Context prefix:** `Koi.Blocklist`

This command takes no arguments.

Outputs

Context path	Type	Description
Koi.Blocklist.item_id	String	The unique identifier of the blocklist item.
Koi.Blocklist.item_name	String	The name of the blocklist item.

Context path	Type	Description
`Koi.Blocklist.item_display_name`	String	The display name of the blocklist item.
`Koi.Blocklist.marketplace`	String	The marketplace of the blocklist item (e.g., vscode).
`Koi.Blocklist.publisher_name`	String	The publisher name of the blocklist item.
`Koi.Blocklist.package_name`	String	The package name of the blocklist item.
`Koi.Blocklist.notes`	String	Notes associated with the blocklist item.
`Koi.Blocklist.created_by`	String	The user who created the blocklist item.
`Koi.Blocklist.created_at`	Date	The creation time of the blocklist item in ISO 8601 format.

6.8 koi-blocklist-items-remove

Potentially harmful — flagged execution: true in the pack

This command changes state in your KOI tenant and is marked as potentially harmful in the pack source. Cortex gates it accordingly. Review the arguments before running it, and be aware that it declares no outputs, so nothing records what it removed.

It was not run during verification. Nothing in this guide describes its observed behaviour.

Removes one or more items from the global blocklist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk removal from a JSON file.

API endpoint: `DELETE /policies/blocklist` · **Context prefix:** `none`

Arguments

Argument	Required	Default	Predefined values	Description
`item_id`	No	—	—	The ID of the item to remove from the blocklist. Required when not using items_list_raw_json_entry_id.
`marketplace`	No	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The source marketplace of the item. Required when not using items_list_raw_json_entry_id.
`created_by`	No	—	—	Email of the user who created this entry.
`notes`	No	—	—	Additional notes about the removal.

Argument	Required	Default	Predefined values	Description
`items_list_raw_json_entry_id`	No	—	—	War Room entry ID of a JSON file containing a list of items to remove. Each item must have "item_id" and "marketplace" fields. Optional fields: "created_by", "notes". When provided, item_id and marketplace arguments are ignored.

Outputs

> **This command declares no outputs**
>
> It writes nothing to the context — it returns a War Room message only. A playbook cannot branch on its result, and a subsequent command cannot read what it did. If you need to confirm the change, follow it with the matching -get command.

6.9 koi-blocklist-items-add

> **Changes state — but is NOT flagged execution: true**
>
> This command modifies governance state in your KOI tenant, and the pack does not mark it as potentially harmful. It therefore runs without the confirmation Cortex applies to flagged commands. Treat it with the same care as the flagged ones.
>
> *It was not run during verification. Nothing in this guide describes its observed behaviour.*

Adds one or more items to the global blocklist. Provide either 'item_id' and 'marketplace' for a single item, or 'items_list_raw_json_entry_id' for bulk addition from a JSON file.

API endpoint: `POST /policies/blocklist` · **Context prefix:** `none`

Arguments

Argument	Required	Default	Predefined values	Description
`item_id`	No	—	—	The ID of the item to add to the blocklist. Required when not using items_list_raw_json_entry_id.
`marketplace`	No	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The source marketplace of the item. Required when not using items_list_raw_json_entry_id.
`created_by`	No	—	—	Email of the user who created this entry.

Argument	Required	Default	Predefined values	Description
notes	No	—	—	Additional notes or justification for blocking the item.
items_list_raw_json_entry_id	No	—	—	War Room entry ID of a JSON file containing a list of items to add. Each item must have "item_id" and "marketplace" fields. Optional fields: "created_by", "notes". When provided, item_id and marketplace arguments are ignored.

Outputs

> **This command declares no outputs**
>
> It writes nothing to the context — it returns a War Room message only. A playbook cannot branch on its result, and a subsequent command cannot read what it did. If you need to confirm the change, follow it with the matching -get command.

6.10 koi-policy-status-update

> **Changes state — but is NOT flagged execution: true**
>
> This command modifies governance state in your KOI tenant, and the pack does not mark it as potentially harmful. It therefore runs without the confirmation Cortex applies to flagged commands. Treat it with the same care as the flagged ones.
>
> *It was not run during verification. Nothing in this guide describes its observed behaviour.*

Enables or disables a policy by ID.

API endpoint: `PUT /policies/{id}` · **Context prefix:** `Koi.Policy`

Arguments

Argument	Required	Default	Predefined values	Description
policy_id	Yes	—	—	The ID of the policy to update.
enabled	Yes	—	true, false	Whether to enable (true) or disable (false) the policy.

Outputs

Context path	Type	Description
Koi.Policy.id	Number	The unique identifier of the policy.
Koi.Policy.name	String	The name of the policy.
Koi.Policy.description	String	The description of the policy.
Koi.Policy.action	String	The action taken by the policy (e.g., block).
Koi.Policy.enabled	Boolean	Whether the policy is enabled.
Koi.Policy.group_ids	Unknown	List of group IDs associated with the policy.

Context path	Type	Description
`Koi.Policy.creator_fullname`	String	The full name of the policy creator.
`Koi.Policy.created_at`	Date	The creation time of the policy in ISO 8601 format.
`Koi.Policy.updated_at`	Date	The last update time of the policy in ISO 8601 format.

6.11 koi-inventory-list

Retrieves a paginated list of items installed across your organization's endpoints. Supports extensive filtering by marketplace, platform, risk level, publisher, and specific categories.

API endpoint: `GET /inventory` · **Context prefix:** `Koi.Inventory`

Arguments

Argument	Required	Default	Predefined values	Description
`page`	No	—	—	Page number for pagination (1-based). When provided, fetches a single page and ignores the limit argument.
`page_size`	No	—	—	Number of results per page (default: 50, max: 500). Used in single-page mode with the page argument.
`limit`	No	50	—	Maximum total number of inventory items to return (default: 50, max: 1000). When provided without page, auto-paginates to collect up to this many items.
`brew_category_koi`	No	—	—	Filter by Homebrew package category (Koi classification).
`browser_category_koi`	No	—	—	Filter by browser extension category (Koi classification).
`chocolatey_category_koi`	No	—	—	Filter by Chocolatey package category (Koi classification).
`device_id`	No	—	—	Filter devices by device ID.
`finding_id`	No	—	—	Filter devices by finding ID.
`first_seen`	No	—	—	Filter by first seen date (items first seen on or after this date). ISO 8601 format (e.g., "2024-01-01T00:00:00Z").
`ide_category_koi`	No	—	—	Filter by IDE extension category (Koi classification).
`installation_method`	No	—	marketplace, manual, built_in, side_loaded	Filter by installation method.
`item_display_name`	No	—	—	Filter by item display name. Performs case-insensitive partial match.
`item_id`	No	—	—	Filter by item ID.

Argument	Required	Default	Predefined values	Description
marketplace	No	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	Filter by marketplace.
platform	No	—	antigravity, aqua, arc, brave, brew, chatgpt_atlas, chocolatey, chrome, chromium, claude, clion, codex, comet, cursor, datagrip, dataspell, dia, edge, excel, firefox, fleet, goland, hugging_face, intellij_community, intellij, kiro, mac, npm, notepad++, opera, outlook, phpstorm, powerpoint, prisma_access_browser, pycharm, pypi, rider, rubymine, rustrover, vscode, webstorm, windsurf, word, windows, writerside	Filter by platform.
publisher_name	No	—	—	Filter by publisher name. Performs case-insensitive partial match.
risk_level	No	—	low, medium, high, critical, pending	Filter by risk level.
software_category_koi	No	—	—	Filter by software category (Koi classification).
sort_by	No	first_seen	first_seen, last_seen, item_display_name, item_id, version, marketplace, endpoint_count, risk, risk_level, status, installs_count, released_at, publisher_name	Column to sort by.
sort_direction	No	—	asc, desc	Sort direction.
view	No	—	agentic_ai, ai_models, code_packages, extensions, os_packages, software	Filter by predefined view (marketplace group).

Outputs

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.

Context path	Type	Description
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

6.12 koi-inventory-item-get

Retrieves comprehensive details for a specific software item, extension, or package using its unique identifier, marketplace, and version.

API endpoint: `GET /inventory/{item_id}` · **Context prefix:** `Koi.Inventory`

Arguments

Argument	Required	Default	Predefined values	Description
item_id	Yes	—	—	Unique identifier for the item.
marketplace	Yes	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The marketplace where the item is hosted.
version	Yes	—	—	The specific version of the item to retrieve.

Outputs

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).

Context path	Type	Description
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item including severity and evidence.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

Argument trap: the marketplace value in an event is not the marketplace value this command wants

`koi-inventory-item-get` and `koi-inventory-item-endpoints-list` both REQUIRE `marketplace`. The value recorded in a `koi_koi_raw` event is not the value these commands accept. Events use short forms (`software_windows`, `chrome`, `vsc`); the argument and the API use long forms (`windows`, `chrome_web_store`, `vscode`). Where they differ, passing the event value straight through returns **HTTP 400**.

`npm` and `pypi` are spelled the same in both; most other high-frequency values are not. Map every event value through the table below before calling either command. Three event values — `built_in`, `side_loaded` and `ollama` — have no API equivalent (the first two are `installation_method` values leaking into the marketplace field). Treat all three as "unknown marketplace" and do not pass them on.

Source: VERIFIED_FACTS.md §7c (live, 21 July 2026).

Event marketplace value → command / API marketplace value

Event value (koi_koi_raw)	Events	Command / API value	Notes
software_windows	5,301	windows	—
pypi	4,674	pypi	unchanged — spelled the same in both
chrome	891	chrome_web_store	—
built_in	829	—	installation_method value in the marketplace field; treat as unknown, do not pass on
npm	775	npm	unchanged — spelled the same in both
software_mac	617	mac	—
homebrew	231	homebrew	—
vsc	175	vscode	—
chocolatey	91	chocolatey	—
cursor	88	cursor	—

Event value (koi_koi_raw)	Events	Command / API value	Notes
github	65	github_mcp_registry	—
edge	48	edge_add_ons	—
firefox	19	firefox_add_ons	—
docker	15	docker	—
npp	12	notepad++	—
openvsx	10	open_vsx_registry	—
jet	5	jetbrains	—
ollama	5	—	no API equivalent; treat as unknown, do not pass on
claude_desktop_extensions	5	claude_desktop_extensions	unchanged — spelled the same in both
side_loaded	1	—	installation_method value in the marketplace field; treat as unknown, do not pass on

The Events column holds 21 July 2026 counts on the reference tenant and only indicates how common each value is — not an expected result. Source: VERIFIED_FACTS.md §7c (live).

6.13 koi-inventory-search

Searches inventory items using advanced query builder filters. Provide a filter via 'filter_json' (inline JSON string) or 'filter_raw_json_entry_id' (War Room file entry ID). At least one filter source must be provided.

API endpoint: `POST /inventory/search` · **Context prefix:** `Koi.Inventory`

Arguments

Argument	Required	Default	Predefined values	Description
filter_json	No	—	—	Advanced filter using query builder syntax as a JSON string. Either filter_json or filter_raw_json_entry_id must be provided.
filter_raw_json_entry_id	No	—	—	War Room entry ID of a JSON file containing the filter object. Takes priority over filter_json when both are provided.
page	No	—	—	Page number for pagination (1-based). When provided, fetches a single page and ignores the limit argument.
page_size	No	—	—	Number of results per page (default: 50, max: 500). Used in single-page mode with the page argument.
limit	No	50	—	Maximum total number of inventory items to return (default: 50, max: 1000). When provided without page, auto-paginates to collect up to this many items.

Argument	Required	Default	Predefined values	Description
sort_by	No	first_seen	first_seen, last_seen, item_display_name, item_id, version, marketplace, endpoint_count, risk, risk_level, status, installs_count, released_at, publisher_name	Column to sort by.
sort_direction	No	desc	asc, desc	Sort direction.

Outputs

Context path	Type	Description
Koi.Inventory.item_id	String	The unique identifier of the inventory item.
Koi.Inventory.item_display_name	String	The display name of the inventory item.
Koi.Inventory.marketplace	String	The marketplace source of the item.
Koi.Inventory.platforms	Unknown	List of platforms where the item is installed.
Koi.Inventory.publisher_name	String	The publisher name of the item.
Koi.Inventory.risk	Number	The numeric risk score of the item.
Koi.Inventory.risk_level	String	The risk level classification of the item.
Koi.Inventory.version	String	The version of the item.
Koi.Inventory.status	String	The governance status of the item.
Koi.Inventory.endpoint_count	Number	The number of endpoints where the item is installed.
Koi.Inventory.installs_count	Number	The total number of installs for the item.
Koi.Inventory.first_seen	Date	The date the item was first seen in ISO 8601 format.
Koi.Inventory.last_seen	Date	The date the item was last seen in ISO 8601 format.
Koi.Inventory.last_used	Date	The date the item was last used in ISO 8601 format.
Koi.Inventory.installation_method	String	The method used to install the item.
Koi.Inventory.short_description	String	A short description of the item.
Koi.Inventory.is_first_party	Boolean	Whether the item is a first-party item.
Koi.Inventory.is_signed	Boolean	Whether the item is signed.
Koi.Inventory.categories	Unknown	List of categories the item belongs to.
Koi.Inventory.findings	Unknown	List of findings associated with the item.
Koi.Inventory.governed_details	Unknown	Governance policy details for the item.
Koi.Inventory.released_at	Date	The release date of the item. Format: YYYY-MM-DD (e.g., 2023-01-15).
Koi.Inventory.brew_category_koi	String	The Homebrew package category (Koi classification).

Context path	Type	Description
Koi.Inventory.browser_category_koi	String	The browser extension category (Koi classification).
Koi.Inventory.chocolatey_category_koi	String	The Chocolatey package category (Koi classification).
Koi.Inventory.ide_category_koi	String	The IDE extension category (Koi classification).
Koi.Inventory.software_category_koi	String	The software category (Koi classification).

6.14 koi-inventory-item-endpoints-list

Retrieves a paginated list of endpoints that have a specific item installed.

API endpoint: `GET /inventory/{item_id}/endpoints` · **Context prefix:** `Koi.Inventory.Endpoint`

Arguments

Argument	Required	Default	Predefined values	Description
item_id	Yes	—	—	Unique identifier for the item.
marketplace	Yes	—	chocolatey, chrome_web_store, claude_desktop_extensions, cursor, docker, edge_add_ons, firefox_add_ons, github_mcp_registry, homebrew, hugging_face, jetbrains, linux, mac, notepad++, npm, office_add_ins, open_vsx_registry, pypi, visual_studio, vscode, windows, windsurf	The marketplace where the item is hosted.
version	Yes	—	—	The specific version of the item.
page	No	—	—	Page number for pagination (1-based). When provided, fetches a single page and ignores the limit argument.
page_size	No	—	—	Number of results per page (default: 50, max: 500). Used in single-page mode with the page argument.
limit	No	50	—	Maximum total number of endpoints to return (default: 50, max: 1000). When provided without page, auto-paginates to collect up to this many endpoints.

Outputs

Context path	Type	Description
Koi.Inventory.Endpoint.id	String	The unique identifier of the endpoint device.

Context path	Type	Description
Koi.Inventory.Endpoint.hostname	String	The hostname of the endpoint.
Koi.Inventory.Endpoint.os	String	The operating system of the endpoint.
Koi.Inventory.Endpoint.platform	String	The platform where the item is installed on this endpoint.
Koi.Inventory.Endpoint.serial	String	The serial number of the endpoint device.
Koi.Inventory.Endpoint.last_logged_on_user	String	The last logged on user of the endpoint.
Koi.Inventory.Endpoint.activation_status	String	The activation status of the endpoint.
Koi.Inventory.Endpoint.path	String	The installation path of the item on the endpoint.
Koi.Inventory.Endpoint.first_seen	Date	The date the item was first seen on this endpoint in ISO 8601 format.
Koi.Inventory.Endpoint.last_seen	Date	The date the item was last seen on this endpoint in ISO 8601 format.

7. The Context Model

7.1 Five prefixes, and the casing is not a typo

The pack writes to 5 context prefixes. Note the inconsistent casing: the event command writes KOI.Event.* in capitals, everything else writes Koi.*. This is what the pack declares. Do not "fix" it in your own content — DT expressions and context paths are case-sensitive, and correcting the case breaks the lookup.

Prefix	Paths	Written by
`KOI.Event`	4	koi-get-events
`Koi.Allowlist`	9	koi-allowlist-get
`Koi.Blocklist`	9	koi-blocklist-get
`Koi.Inventory`	37	koi-inventory-list, koi-inventory-item-get, koi-inventory-search, koi-inventory-item-endpoints-list
`Koi.Policy`	9	koi-policy-list, koi-policy-status-update

Koi.Inventory.Endpoint. is nested beneath Koi.Inventory and is counted within it above; it is the only place endpoints appear. That nesting is why four commands are listed against Koi.Inventory: koi-inventory-item-endpoints-list writes only the 10 nested Endpoint.* paths and shares no context path with the other three (see 7.3).*

7.2 The model is item-centric — there is no device object

> **Koi.Device.* does not exist in this pack**
>
> There is no device-listing command and no device context prefix. The count of 'Koi.Device.' occurrences is zero in the integration YAML, zero in Koi.py and zero in the integration README.
>
> Endpoints are reached only from an item: run `koi-inventory-item-endpoints-list` with an item_id, marketplace and version, and read `Koi.Inventory.Endpoint.*`. You cannot start from a host and enumerate what it runs — the traversal only goes item → endpoints.
>
> *Source: VERIFIED_FACTS.md §4.*

If you need a per-device view, you must build it yourself by iterating items and grouping the endpoint results — accepting that this only ever covers items you have already listed.

7.3 Three inventory commands write the same 27 paths — the last one wins

The pack declares 131 output declarations but only 68 distinct context paths. The difference, 63, is a count of redundant declarations, not of paths: 36 distinct paths are declared by more than one command. `koi-inventory-list`, `koi-inventory-item-get` and `koi-inventory-search` — three commands, not four — declare an identical set of 27 Koi.Inventory.* item paths, and `koi-policy-list` and `koi-policy-status-update` share all nine Koi.Policy.* paths. 27 + 9 = the 36 shared paths.

koi-inventory-item-endpoints-list is not part of this collision. It appears under Koi.Inventory in the table in 7.1 only because `Koi.Inventory.Endpoint.*` is nested there. It writes those 10 nested paths and

nothing else, and shares no context path with the other three, so it neither overwrites them nor is overwritten by them.

> **Ordering hazard in playbooks**
>
> Because those three commands write the same 27 paths, the last one to run overwrites the previous one's context. A playbook that lists inventory, then fetches one item, then reads Koi.Inventory expecting the list gets the single item instead.
>
> Two ways to avoid it: read the context immediately after the command that produced it and copy it into your own key (for example with Set), or do not mix the three in one playbook branch.

7.4 Four commands write nothing at all

4 commands declare no outputs: `koi-allowlist-items-remove`, `koi-allowlist-items-add`, `koi-blocklist-items-remove`, `koi-blocklist-items-add`. They return a War Room message only. A playbook cannot test whether they succeeded from context, and a downstream task has nothing to consume. To confirm the result, call `koi-allowlist-get` or `koi-blocklist-get` afterwards and compare the list.

8. Event Collection and Querying

This whole section applies to Cortex XSIAM only. The pack's event collector is disabled for the xsoar marketplace at the source level (section 2.1), so on an XSOAR tenant there is no collector, no dataset and nothing to query.

8.1 The dataset

On XSIAM, with Fetch events enabled, KOI events land in the dataset **`koi_koi_raw`**, with `_vendor = "koi"` and `_product = "koi"`. This was confirmed on the live tenant: over a 30-day window (20 June – 20 July 2026) the dataset held 20,156 events across 80 distinct hostnames. The pack never declares the dataset name anywhere in its content — it follows from the vendor/product pair the collector sends.

8.2 One dataset, two incompatible schemas

The field `source_log_type` is the discriminator. Audit rows and Alert rows share nothing: every OCSF field is null on Audit rows, and every Audit field is null on Alert rows. Treat them as two datasets that happen to share a name.

source_log_type	Count (30 days)	Schema
`Audit`	19,842	Flat, KOI-native: type, action, category, object_name, object_type, hostname, item_version.
`Alerts`	314	OCSF: class_uid 2007 ("Application Security Posture Finding"), type_uid 200701, is_alert true, severity_id, confidence_id, risk_level_id, status_id.

Observed Audit type values and counts over that window: extensions 16,579 · devices 2,971 · remediation 244 · policies 32 · approval_requests 8 · guardrails 6 · notifications 2. Observed category values: system 16,825 · user 3,017.

Counts are from the reference tenant over 20 June – 20 July 2026 and are illustrative of shape, not of your volume. Source: VERIFIED_FACTS.md §3.

Two traps that make queries return nothing

1. On alert rows, `resources`, `observables` and `metadata` are JSON strings, not objects. A query that treats them as structured fields silently returns nothing — no error, just no rows. You must extract from the string.

2. `alert_type` is never populated. A filter of alert_type != null over the full 30-day window returned zero rows. Any query, correlation rule or playbook keyed on alert_type matches nothing here.

Source: VERIFIED_FACTS.md §3.1 (live).

8.3 The Alerts stream is duplicated — every alert count must dedupe

Counting Alert rows overcounts by hundreds of times — read this before writing any alert query

This is the single most important thing to know before you query alerts. **The integration re-sends every still-open alert on every fetch cycle**, so `koi_koi_raw` holds one row PER ALERT PER FETCH, not one row per alert. With `eventFetchInterval = 1` minute, a single open alert becomes hundreds of identical rows over a day (357 rows sharing one `_time` and one message, with 357 distinct `_insert_time` values, were observed inside one notification).

Over the last 24 hours on the reference tenant (21 July 2026), the Alerts stream held **734 rows for just 3 distinct alerts — 244.7× inflation**. **Audit is NOT affected**: each audit record is point-in-time and carries a unique KOI `id` (257 rows / 257 distinct over the same 24 h). This is an Alerts-only problem.

Source: VERIFIED_FACTS.md §7e (live, 21 July 2026).

Stream	Window	Rows	Distinct alerts	Inflation
Alerts	last 24 h	734	3	244.7×
Alerts	last 90 d	1,048	317	3.3×
Audit	last 24 h	257	257 (by KOI id)	1.0 — none
Audit	last 90 d	20,148	20,148	1.0 — none

Figures are from the reference tenant on 21 July 2026 and are illustrative of shape, not of your volume. Source: VERIFIED_FACTS.md §7e (live).

The only correct dedupe key is `metadata.notification_event_id`. Over the 90-day window it has 317 distinct values across 1,048 alert rows — a verified 1:1 identity for a single alert occurrence. The other candidate identifiers are wrong at both extremes, so do not reach for them:

Field	Distinct / 1,048 rows	What it identifies
`_id`	1,048	the row — counts every duplicate
`metadata.notification_event_id`	317	the alert occurrence — use this
`observables[event.id] (koi_event_id)`	20	the scan batch — far too coarse
`finding_info.uid (finding_uid)`	3	the finding / policy definition — far too coarse

Any query, widget, correlation rule or playbook that counts alerts must dedupe on this key. Because the pack ships no parsing rule, there is no promoted column — extract it inline from the raw metadata field. This is the corrected count:

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| alter koi_notification_id = json_extract_scalar(metadata,
"$.notification_event_id")
| comp count_distinct(koi_notification_id) as alerts
```

On the reference data for the last 24 hours (21 July 2026) this returns **3**, where the naive `comp count() as alerts` returns **734**. A triage playbook that reacts per alert row will likewise fire hundreds of times for one real alert unless it dedupes on the same key.

Source: VERIFIED_FACTS.md §7e (live, 21 July 2026). The XQL is constructed from the verified field name and was not itself re-executed on the tenant.

8.4 No XDM — query the raw fields

The pack ships no parsing rule and no modeling rule, so nothing maps this data to the Cortex Data Model. No XDM field is populated. Every query below targets raw field names deliberately.

8.5 XQL starting points

Confirm the collector is delivering, and see the split:

```
dataset = koi_koi_raw
| comp count() as events by source_log_type
```

Most recent rows, whichever stream:

```
dataset = koi_koi_raw
| sort desc _time
| limit 20
```

Audit activity broken down by type and category:

```
dataset = koi_koi_raw
| filter source_log_type = "Audit"
| comp count() as events by type, category
| sort desc events
```

Audit events for one host:

```
dataset = koi_koi_raw
| filter source_log_type = "Audit" and hostname = "<HOSTNAME>"
| fields _time, type, action, category, object_name, object_type, item_version
| sort desc _time
```

Alerts, with the OCSF fields that are actually populated:

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| fields _time, class_uid, type_uid, severity_id, confidence_id, risk_level_id,
status_id
| sort desc _time
```

Alert severity distribution — count_distinct on the dedupe key, not count(), because the Alerts stream is duplicated (8.3):

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| alter koi_notification_id = json_extract_scalar(metadata,
"$.notification_event_id")
| comp count_distinct(koi_notification_id) as alerts by severity_id
| sort desc alerts
```

Reading the JSON-string fields on alert rows — this is the form that works:

```
dataset = koi_koi_raw
| filter source_log_type = "Alerts"
| alter resources_obj = json_extract(resources, "$")
| alter metadata_obj  = json_extract(metadata, "$")
```

```
| fields _time, resources_obj, metadata_obj
```

The field names and the JSON-string behaviour above are verified (VERIFIED_FACTS.md §3). The XQL statements themselves are constructed from those field names for this guide and were not each re-executed on the tenant; adjust the extraction path inside json_extract to the element you need.

9. Known Issues, Gaps and Behaviours to Know About

This section has five subsections holding seven findings, of four different kinds — defects, a documentation gap, a response-shape observation and behaviours worth knowing — not seven defects. Two are defects in the pack as published: 9.1, the incomplete view dropdown, and 9.2, the broken shipped command example. One is a documentation gap rather than a fault: 9.3 — the filter shape works, the pack simply never illustrates it. One is an observation about the API's response shape that a runbook can trip over, not a fault at all: 9.4. The remaining three are behaviours worth knowing, collected in 9.5: a property of the pack's own metadata (three state-changing commands are not flagged harmful), an authentication observation, and a repeatability caveat. Nothing here is a misconfiguration on your side, and nothing outside 9.1 and 9.2 is something the pack gets wrong.

9.1 The view dropdown is missing three valid values

The `view` argument of `koi-inventory-list` offers 6 predefined values: `agentic_ai, ai_models, code_packages, extensions, os_packages, software`. The API accepts nine. Its own 400 response states the contract: agentic_ai, ai_models, all_items, code_packages, extensions, mcp_servers, os_packages, repositories, software.

All twelve values were probed individually against the live API on both reference instances — the nine the API names, plus three a reader might plausibly guess. Nothing about the view argument is left untested, and no result below is inferred from another. The table below shows seven of the twelve: the three the dropdown omits, the three the API rejects, and software as a control. The five it leaves out are dropdown values the API accepts — agentic_ai, ai_models, code_packages, extensions and os_packages — and each returned HTTP 200 with a non-zero total_count on both instances, from 3 (ai_models on KOI_PAET) to 2,572 (code_packages on KOI_PLTS).

view value	In dropdown	Live result	What it means
`mcp_servers`	No	HTTP 200	The MCP-server audit case. Returned 21 items on KOI_PAET and 42 on KOI_PLTS. Works when typed by hand; the dropdown will never offer it.
`repositories`	No	HTTP 200	Returned 15 items on KOI_PAET and 77 on KOI_PLTS. Repository inventory is unreachable from the dropdown.
`all_items`	No	HTTP 200	Accepted, but returned total_count 0 on BOTH instances, while every other accepted value returned data. Its name promises everything and it delivered nothing. To list everything, omit view entirely — that returned 3,447 items on KOI_PAET, and 5,646 on KOI_PLTS on the later of two sweeps that day, up from 5,644 hours earlier (see the drift note below).
`software`	Yes	HTTP 200	Returned 406 items on KOI_PAET and 1,046 on KOI_PLTS in this probe run — the KOI_PLTS figure was 1,044 a few hours earlier the same day and is drifting upward. Shown for contrast: a dropdown value behaving normally.
`browser_extensions`	No	HTTP 400	Not a valid value. This is the one that appears in the pack's shipped example — see 9.2.
`ide_extensions`	No	HTTP 400	Not a valid value. Does not appear anywhere in the pack; listed here because it is a plausible guess.
`packages`	No	HTTP 400	Not a valid value. Does not appear anywhere in the pack; listed here because it is a plausible guess.

Nothing in the pack is invalid — the list is merely incomplete. Workaround: type the value into the argument by hand instead of selecting it; the API accepts the three missing values. Sources: the predefined list above is generated from the pack source; the probe results are from VERIFIED_FACTS.md §5.1 (live).

> **The counts above are a snapshot, not a threshold**
>
> Every number in this table is a single reading taken on the reference tenant on 20 July 2026, and KOI inventory can grow underneath you: two sweeps a few hours apart the same day returned different totals on KOI_PLTS — view=software 1,044 then 1,046, and the whole inventory 5,644 then 5,646. KOI_PAET did not move: it returned 3,447 in both sweeps. Re-probing twice inside a single run returned identical values every time on both instances, so this is real growth on one named tenant, not an unstable API.
>
> Do not use any figure here as a pass/fail threshold. What is stable, and what you should check for, is the shape of the response: HTTP 200, a total_count present and non-zero for a value the API accepts, HTTP 400 for one it does not. A count that differs from this table — on your tenant or on this one an hour later — is not a failed test.

9.2 The pack's own command example is broken

The file `Packs/Koi/Integrations/Koi/command_examples.txt` in the pack contains `view=browser_extensions`. That value is in neither the pack's own predefined list nor the API's accepted set, and the live API rejects it with HTTP 400. Anyone copying the shipped example gets an error.

Only browser_extensions is in the shipped example. `ide_extensions` and `packages` also return HTTP 400, but they do not appear in `command_examples.txt` or anywhere else in the pack — they are listed in 9.1 only because they are values a reader might guess. Use the nine values the API named in 9.1.

Sources: the broken value is read from the pack's own command_examples.txt; the HTTP 400 it produces is from VERIFIED_FACTS.md §5.2 (live).

9.3 koi-inventory-search needs a structured filter, and the pack never shows its shape

The `filter_json` argument is described in the pack as query-builder syntax, but the pack gives no example of what that syntax looks like. What is missing is an example, not a usable error message — the errors are clear. The API requires the combinator/rules form:

```
{"combinator": "and", "rules": [{"field": "risk_level", "operator": "=", "value": "high"}]}
```

That exact filter is verified working: sent in a direct POST to /inventory/search on KOI_PAET — not through the command, which was never executed here — it was accepted and returned a total_count of 145. Treat the 145 as a snapshot of that tenant at that moment, not as an expected result.

Three different things go wrong in three different ways. They are easy to confuse, so they are separated here:

What you do	Result	What you see
Call the command with neither filter_json nor filter_raw_json_entry_id	No API call	The integration itself stops you: "Either 'filter_json' or 'filter_raw_json_entry_id' must be provided." Nothing reaches the API. This row is read from the integration's Koi.py source, not reproduced — no koi-* command was executed through Cortex at any point (section 5).
Supply a filter that is present but malformed, for example {}	API 400	The response body names the problem: filter.combinator must be one of the following values: and, or; filter.rules must be an array. This 400 is not opaque — it tells you which keys are wrong. Reproduced by direct API request on both instances.
Omit the filter key from a request made to the API directly	API 500	Internal Server Error, reproduced by direct API request on both instances. Reachable only by calling the API yourself; the command cannot produce this, because the check above catches it first.

Sources: the two API results are from VERIFIED_FACTS.md §5.3 (live, both instances); the argument check in the first row is from the integration source code, which was read and not executed.

The same JSON can be supplied as a War Room file entry instead, via `filter_raw_json_entry_id`, which avoids quote-escaping problems in the CLI.

9.4 Allowlist and blocklist responses carry no total

The allowlist and blocklist endpoints return only an `items` array — no `total_count`, unlike the policies and inventory endpoints. Any runbook step that tells an operator to check total_count on `koi-allowlist-get` or `koi-blocklist-get` is pointing at a field that never exists. Count the returned items instead.

Related: an empty items array is the correct, non-error result for an empty list — both lists were empty on KOI_PAET (KOI_PLTS held 5 allowlist and 17 blocklist entries). Do not read it as a failure. Source: VERIFIED_FACTS.md §5.4 (live).

9.5 Three behaviours worth knowing

- **Three of the five state-changing commands are not flagged as harmful.** Only `koi-allowlist-items-remove` and `koi-blocklist-items-remove` carry execution: true. `koi-policy-status-update`, `koi-allowlist-items-add`, `koi-blocklist-items-add` change tenant state with no such guard (section 6.1).
- **A bad API key fails cleanly.** An invalid bearer token returns HTTP 401 with the body `{"message":"Unauthorized","statusCode":401}`, verified with a deliberately invalid key. If you see 401, suspect the key. No 403 was ever observed from the reference environment, so this guide makes no claim about what a 403 would mean here.
- **koi-get-events is a debugging tool, not a collector, and it is the one read command that is not safe to repeat.** The pack's own description says it is for development and debugging only, as it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism. Its `should_push_events` argument writes the retrieved events into `koi_koi_raw` when true, so a careless re-run duplicates rows in the same dataset the collector is filling. Leave it at its default of

`false`, and treat the other 7 non-mutating commands — not this one — as the freely repeatable set (section 6.1).

10. Provenance of Every Claim in This Guide

This guide was built to a method: every factual claim about the pack is generated at build time from the pinned pack source, read from the integration's own source code, or observed live, and the sources are named section by section rather than sentence by sentence. Nothing was written from memory of a similar pack. The method does not amount to a guarantee that every individual sentence carries a tag — some connective and explanatory statements (how the Cortex platform behaves in general, why a collision has the consequence it does) rest on none of the three, and 10.1 gives the main examples of those.

Marking	Meaning
Pack source	Generated at build time from Packs/Koi/Integrations/Koi/Koi.yml (md5 5497cdddedeb0c0d7d0b371aa075a64c), confirmed byte-identical to demisto/content@master on 2026-07-20. This covers every command, argument, default, predefined value, context path, configuration parameter, version and the docker image — every table in sections 4 and 6 is produced by code from that file. The per-command API endpoint line in section 6 is the one thing there that is not: see the Integration source row below.
Live	Observed on tenant api-ayman.xdr.eu.paloaltonetworks.com on 20 July 2026 (instances KOI_PAET and KOI_PLTS), or against the KOI API with the same keys — never through a Cortex War Room, which the tenant would not allow. Covers the dataset name, the event schema split, the findings in sections 9.1 to 9.4 and the 401 observation in 9.5. It covers the API behind 8 of the 13 commands only; the 5 state-changing commands were not run at all. Live counts are snapshots that drift (see 9.1) and are never expected results.
Integration source	The endpoint map (section 6) is read from the integration's Koi.py, which is the pack's source code — not the YAML, and not by itself an observation. The map has 11 entries; 8 of them were then also exercised live, and the other 3 — PUT /policies/{id}, the POST and DELETE calls on /policies/allowlist, and the POST and DELETE calls on /policies/blocklist, which are the endpoints behind the 5 state-changing commands — were never called. The same applies to the argument check in the first row of the 9.3 table.
Not from any of those	The main cases are listed below. That list is illustrative of the kinds of statement involved, not a complete inventory of every sentence in the guide.

10.1 Main statements not traceable to the pack source, the integration source or live verification

These are the recurring kinds. They are general platform behaviour or reasoning about consequences, not observations of this pack. Other sentences of the same kind exist elsewhere in the guide; this table names the ones a reader is most likely to want to weigh.

Statement	Status
Cortex UI navigation wording in section 3 (Marketplace, search, install)	Standard platform navigation, not part of the pack and not re-verified for this document. Flagged in place in section 3.
The XQL statements in section 8.5	Constructed for this guide from verified field names and verified JSON-string behaviour; the statements themselves were not each executed. Flagged in place in section 8.5.
"Cortex will require confirmation to run execution: true commands" (section 6)	Platform behaviour for commands flagged execution: true. The flag itself is from the pack source; the platform's handling of it is general product behaviour, not verified on the reference tenant.
"Because Cortex keys content on those identifiers, the two packs cannot be installed side by side" (section 1.2)	The shared identifiers are from the pack source. That Cortex resolves a collision by overwriting is platform behaviour; no overwrite was performed on the reference tenant to watch it happen.
The playbook ordering hazard in section 7.3 and the "last one wins" consequence	Deduced from the declared context paths, which are from the pack source. No playbook was built and run to observe the overwrite.
The remediation advice throughout (workarounds, what to do instead)	Written for this guide. It follows from the verified behaviour but is not itself a verified observation.

10.2 The one gap in the evidence

No command was ever executed through Cortex, because the reference tenant blocks the API paths the tooling needs (section 5). No War Room output, no human-readable table and no context mapping was observed for any of the 13 commands; those are asserted from the pack's own files alone — the YAML for the declarations, Koi.py for the endpoints and the argument checks. Underneath that, the API behind 8 of the 13 commands was exercised directly, and the 5 state-changing commands were not run at all — so for those five, neither layer was observed. To close the gap, run the commands from the XSIAM UI War Room, which does not use the failing API path.

10.3 Rebuilding this document

The generator reads the pinned JSON at run time and writes the same bytes on every run. If the pack is updated upstream, regenerate the source JSON first — do not edit the tables by hand.

```
export NODE_PATH=<repo>/node_modules
node docs/build_guide.js
```